

Lab 1: Introduction to NUnit

Objective

Set up a basic NUnit project and write simple tests.

Prerequisites

- Visual Studio (2019 or later) installed.
- Basic understanding of C# and object-oriented programming.

Instructions

Step 1: Set Up the Project

1. Open Visual Studio and create a new **Class Library (.NET)** project.
2. Name the project `NunitLab`.
3. Add a second project to the solution for unit tests:
 - Right-click on the solution in **Solution Explorer**, choose **Add > New Project**.
 - Select **Unit Test Project (.NET)** and name it `NunitLab.UnitTests`.
4. Add a reference to the main project from the test project:
 - Right-click on `NunitLab.UnitTests`, choose **Add > Project Reference**.
 - Select `NunitLab` and click **OK**.

Step 2: Install NUnit

1. In the `NunitLab.UnitTests` project, open the **NuGet Package Manager**:
 - Right-click on the project, choose **Manage NuGet Packages**.
2. Search for and install the following packages:
 - **NUnit**
 - **NUnit3TestAdapter**

NuGet: NunitLab.UnitTests

Browse Installed Updates

Search (Ctrl+L)   ☐ Include prerelease ☐ Show only vulnerable

Top-level packages (5)

-  **coverlet.collector** by tonerdo 6.0.2
Coverlet is a cross platform code coverage library for .NET, with support for line, branch and method coverage.
-  **Microsoft.NET.Test.Sdk** by Microsoft 17.12.0
The MSbuild targets and properties for building .NET test projects.
-  **NUnit** by Charlie Poole, Rob Prouse 4.2.2
NUnit is a unit-testing framework for all .NET languages with a strong TDD focus.
-  **NUnit.Analyzers** by NUnit 4.4.0
Code analyzers and fixes for NUnit 3+
-  **NUnit3TestAdapter** by Charlie Poole, Terje Sandstrom 4.6.0
NUnit3 adapter for running tests in Visual Studio and DotNet. Works with NUnit 3.x, use the NUnit 2 adapter for 2.x tests.

Transitive packages (5)

Step 3: Write Unit Tests

1. In the `NunitLab.UnitTests` project, create a new test class `CalculatorTests.cs`:

```
using NUnitLab.Api;

namespace NUnitLab.UnitTests;

[TestFixture]
public class CalculatorTests
{
    private Calculator? _systemUnderTest;

    public Calculator SystemUnderTest
    {
        get
        {
            if (_systemUnderTest == null)
            {
```

```

        _systemUnderTest = new Calculator();
    }

    Assert.That(_systemUnderTest, Is.Not.Null);

    return _systemUnderTest;
}

[TearDown]
public void TearDown()
{
    _systemUnderTest = null;
}

[Test]
public void Add_ShouldReturnCorrectSum()
{
    // arrange
    var value1 = 2;
    var value2 = 3;
    var expected = 5;

    // act
    var actual = SystemUnderTest.Add(value1, value2);

    // assert
    Assert.That(actual, Is.EqualTo(expected));
}

[Test]
public void Subtract_ShouldReturnCorrectDifference()
{
    // arrange
    var value1 = 2;
    var value2 = 3;
    var expected = -1;

    // act
    var actual = SystemUnderTest.Subtract(value1, value2);

    // assert
    Assert.That(actual, Is.EqualTo(expected));
}

[Test]
public void Multiply_ShouldReturnCorrectProduct()
{
    // arrange

```

```

    var value1 = 2;
    var value2 = 3;
    var expected = 6;

    // act
    var actual = SystemUnderTest.Multiply(value1, value2);

    // assert
    Assert.That(actual, Is.EqualTo(expected));
}

[Test]
public void Divide_ShouldReturnCorrectQuotient()
{
    // arrange
    var value1 = 6;
    var value2 = 3;
    var expected = 2;

    // act
    var actual = SystemUnderTest.Divide(value1, value2);

    // assert
    Assert.That(actual, Is.EqualTo(expected));
}
}

```

The code should look something like the screenshot below.

```

CalculatorTests.cs
NunitLab.UnitTests
1  using NunitLab.Api;
2
3  namespace NunitLab.UnitTests;
4
5  [TestFixture]
6  public class CalculatorTests
7  {
8      private Calculator? _systemUnderTest;
9
10     public Calculator SystemUnderTest
11     {
12         get
13         {
14             if (_systemUnderTest == null)
15             {
16                 _systemUnderTest = new Calculator();
17             }
18
19             Assert.That(_systemUnderTest, Is.NotNull);
20

```

```
20
21     return _systemUnderTest;
22 }
23
24
25 [TearDown]
26 0 references | 0 changes | 0 authors, 0 changes
27 public void TearDown()
28 {
29     _systemUnderTest = null;
30 }
31
32 [Test]
33 0 references | 0 changes | 0 authors, 0 changes
34 public void Add_ShouldReturnCorrectSum()
35 {
36     // arrange
37     var value1 = 2;
38     var value2 = 3;
39     var expected = 5;
40
41     // act
42     var actual = SystemUnderTest.Add(value1, value2);
43
44     // assert
45     Assert.That(actual, Is.EqualTo(expected));
46 }
47
48 [Test]
49 0 references | 0 changes | 0 authors, 0 changes
50 public void Subtract_ShouldReturnCorrectDifference()
51 {
52     // arrange
53     var value1 = 2;
54     var value2 = 3;
55     var expected = -1;
```

Step 4: Try to Compile

1. From the main menu, choose **Build > Build Solution...**

It won't compile because there's no implementation

Step 5: Implement Enough to Compile

Let's get the compilation working

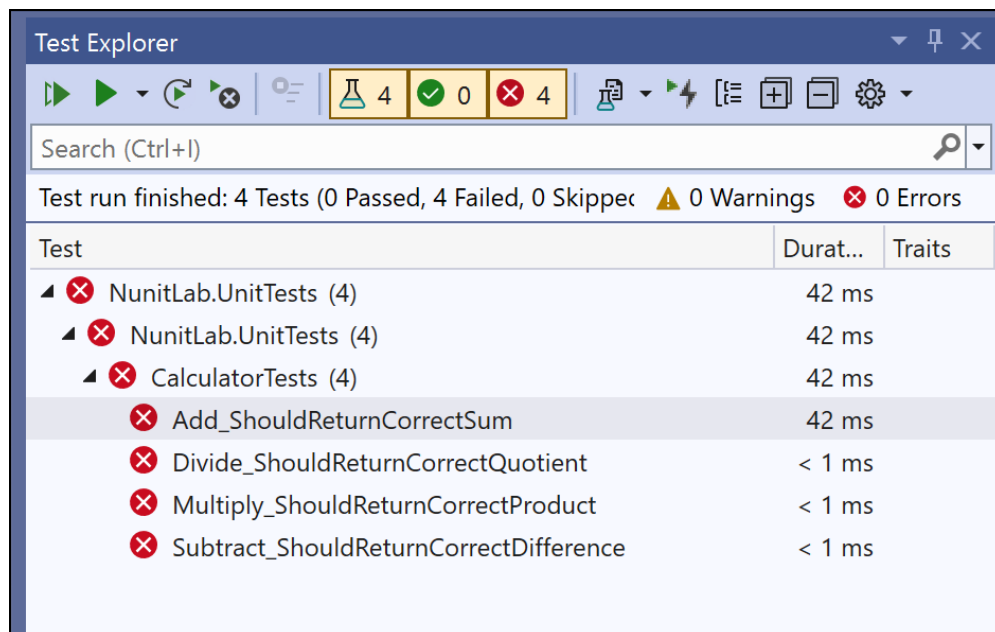
1. In the `NunitLab.Api` project, create a class called `Calculator.cs`:

```
namespace NunitLab.Api
{
    public class Calculator
    {
        public int Add(int a, int b) => default;
        public int Subtract(int a, int b) => default;
        public int Multiply(int a, int b) => default;
        public int Divide(int a, int b) => default;
    }
}
```

NOTE: you could also make these methods `throw new NotImplementedException()` if you wanted

Step 8: Run the Tests

1. Open the **Test Explorer** in Visual Studio:
 - Go to **Test > Test Explorer**.
2. Build the solution, and your tests will appear in the **Test Explorer**.
3. Run all tests and verify they (hopefully) fail.



Step 9: Implement Enough to Make the Tests Pass

1. In the `NunitLab.Api` project, create a class called `Calculator.cs`:

```

namespace NunitLab.Api
{
    public class Calculator
    {
        public int Add(int a, int b) => a + b;
        public int Subtract(int a, int b) => a - b;
        public int Multiply(int a, int b) => a * b;
        public int Divide(int a, int b) => a / b;
    }
}

```

Step 10: Run the Tests

1. Open the **Test Explorer** in Visual Studio:
 - Go to **Test > Test Explorer**.
2. Build the solution, and your tests will appear in the **Test Explorer**.
3. Run all tests and verify they (hopefully) pass.

